Agent tools

Run Python code

Copy MCP Server

Reference information for using the Run Python code tool in Atlas AI agents, including supported parameter types, code structure requirements, and best practices

The **Run Python code** tool enables Atlas AI agents to run custom Python scripts with runtime arguments. The tool automatically validates your code, generates contracts for the agent from Python type annotations, and provides testing capabilities within the Atlas AI interface.

Required code structure

To ensure your script works with the Atlas AI agent, you must define specific entry points and follow the required function signatures.

The handle() function

Every Python script must include a `handle()` function that serves as the entry point. The function must be named `handle` and must have type annotations for all parameters. The function must also include a docstring following the **Google Python Style Guide**.

```
    Args:
        param1 (str): Description of the first parameter.
        param2 (int): Description of the second parameter.

    Returns:
        str: Description of what is returned.
    """
    # Your implementation here
    return f"Processed {param1} with value {param2}"
```

Requirements:

- The function must be named `handle`.
- All parameters must have type annotations. See **Supported parameter types**.
- The function should include a docstring following the **Google Python Style Guide**.
- The return type annotation is recommended but not required.
- The function can be synchronous (`def handle()`) or asynchronous (`async def handle()`).

> ⚠ Only the parameter descriptions in the `Args:` section of the docstring are used by the AI agent. The function description and return value description are not passed to the agent. To provide descriptions for the agent on how to use the tool, use the **Instructions** field in the agent configuration interface.

Async handle() function

You can define `handle()` as an async function using `async def` to perform asynchronous operations.

```
    await asyncio.sleep(0.1)
    return value * 2
```

When to use async handle:

- **Eventual consistency**: When you create a resource and need to wait before retrieving it to ensure it's available.
- **Delayed operations**: When you need to introduce delays between operations.
- **Async SDK operations**: When using async methods from libraries that support async/await patterns.

Requirements for async handle:

- Use `async def` instead of `def` for the function definition.
- Use `await` for asynchronous operations like `asyncio.sleep()`.
- All the same requirements apply as for synchronous `handle()` functions (type annotations, docstrings, etc.).
- The same parameter types and return types are supported for both sync and async functions.

The test() function

Include a `test()` function for validation and testing. Use the **Test code** button in the Atlas AI agent builder to trigger the function.

```
    # Or return a value to display in the console
    return handle("example", 100)
```

Requirements:

- The function must be named `test`.
- No parameters are allowed.
- The function can be synchronous (`def test()`) or asynchronous (`async def test()`).
- The function can use assertions or return values for display.
- The function is run when you click the **Test code** button in the UI.

Async test() function

You can define `test()` as an async function to test async operations or await async `handle()` functions:

```
    await asyncio.sleep(0.1)
    return value * 2

async def test():
    """Async test function that tests async handle."""
    # Await the async handle function directly
    result = await handle(21)
    assert result == 42, f"Expected 42, got {result}"

    return result
```

When to use async test:

- When testing async `handle()` functions you can await them directly.
- When you need to perform async operations in your test setup or teardown.
- When testing scenarios that involve delays or eventual consistency.

Supported parameter types

Parameters can be of primitive types, `DateTime`, `NodeId` and lists of these. Primitive types can have a default value.

Primitive types

Python type	JSON schema type	Example usage	Runtime value
`str`	`"string"`	`name: str`	`"John Doe"`
`int`	`"integer"`	`age: int`	`25`
`float`	`"number"`	`temperature: float`	`23.5`

```python
def handle(
    name: str,
    age: int,
    temperature: float,
    is_active: bool
) -> dict:
    """
    Example with primitive types.

    Args:
        name (str): User's name.
        age (int): User's age in years.
        temperature (float): Temperature in Celsius.
        is_active (bool): Whether user is active.

    Returns:
        dict: Processed user data.
    """
    return {
        "name": name,
        "age": age,
        "temperature": temperature,
        "is_active": is_active
    }
```

Default values

Primitive types support default values:

```
    enabled: bool = True
) -> str:
    """
    Example with default values.

    Args:
        message (str): The message to process.
        count (int): Number of repetitions (default=1).
        multiplier (float): Multiplier factor (default=1.0).
        enabled (bool): Whether processing is enabled (default=True).

    Returns:
        str: Processed message.
    """
    if not enabled:
        return "Processing disabled"

    return (message * count) * int(multiplier)
```

List types

We support lists of primitive types:

```
) -> dict:
    """
    Example with list parameters.

    Args:
        names (list[str]): List of names.
        values (list[int]): List of integer values.
        temperatures (list[float]): List of temperature readings.

    Returns:
        dict: Summary of the lists.
    """
    return {
        "name_count": len(names),
        "total_value": sum(values),
        "avg_temperature": sum(temperatures) / len(temperatures) if temperatures e
    }
```

⚠ We don't support nested lists (e.g., `list[list[str]]`).

DateTime type

For temporal data use `datetime`:

```
    end_time: datetime.datetime,
    timestamps: list[datetime.datetime]
) -> dict:
    """
    Example with datetime parameters.

    Args:
        start_time (datetime.datetime): Start of time range.
        end_time (datetime.datetime): End of time range.
        timestamps (list[datetime.datetime]): List of timestamps.

    Returns:
        dict: Time range information.
    """
    duration = end_time - start_time
    return {
        "duration_seconds": duration.total_seconds(),
        "timestamp_count": len(timestamps)
    }
```

NodeId type

To work with Cognite Data Fusion (CDF) instances from data models, use `NodeId` to identify the instances:

```
    Example with NodeId parameters.

    Args:
        asset_id (NodeId): Primary asset identifier with space and external_id.
        related_asset_ids (list[NodeId]): List of related asset identifiers.

    Returns:
        dict: Asset information.
    """
    # NodeId objects have .space and .external_id properties
    return {
        "primary_asset": {
            "space": asset_id.space,
            "external_id": asset_id.external_id
        },
        "related_count": len(related_asset_ids)
    }
```

Runtime format: `NodeId` parameters are passed to the language model as objects with `space` and `externalId` properties:

```
{
  "space": "my-space",
  "externalId": "asset-123"
}
```

> The `NodeId` class uniquely identifies nodes in Cognite's data modeling framework, combining a `space` (workspace identifier) and `external_id` to form a unique node reference. See the for details.

Limitations and restrictions

- Complex generics beyond `list[T]`
- Union types (`str | int`)
- Optional types (`Optional[str]`)
- Tuples, sets, and other collections
- String literal types (e.g., `Literal["A"]` , `Literal["A", "B"]`)

The following restrictions apply for default argument values for `handle()` :

- Default values are only supported for primitive types (`str` , `int` , `float` , `bool`).
- Lists cannot have default values.
- `NodeId` and `datetime` parameters cannot have default values.

Supported return types

Your `handle()` function can return any JSON-serializable data. The following return types are supported:

Primitive return types

```
    return 42

def handle() -> float:
    return 3.14159

def handle() -> bool:
    return True
```

Dictionary returns

```
def handle() -> dict:
    return {
        "status": "completed",
        "count": 10,
        "results": ["item1", "item2"]
    }
```

List returns

```
def handle() -> list[str]:
    return ["result1", "result2", "result3"]

def handle() -> list[dict]:
    return [
        {"id": 1, "name": "Item 1"},
        {"id": 2, "name": "Item 2"}
    ]
```

None/null returns

Unsupported return types

The following return types are **not supported**:

- Raw Cognite SDK objects
- Custom class instances
- Functions or callable objects
- File handles or binary data
- Complex objects that can't be serialized to JSON

Cognite SDK and authentication

The Cognite Python SDK is pre-installed and automatically configured with your session credentials. The current user credentials are used and no additional configuration is needed.

```
    """
    List assets.

    Returns:
        dict: Asset information including ID and metadata.
    """
    # Client is automatically configured - no setup required
    client = CogniteClient()

    # List assets
    assets = client.data_modeling.instances.list(sources=
[CogniteAsset.get_source()])

    if assets:
        return assets[0].dump()  # Use .dump() for JSON serialization

    return {"error": "No assets found"}
```

For complete SDK usage and API reference, see the **Cognite Python SDK documentation**.

Cognite SDK return types

We don't support direct serialization of Cognite SDK objects. To reduce the number of LLM tokens the result will require and to make the data understandable to the agent, we recommend extracting relevant values and potentially reshaping the data.

```
    """
    Get simplified asset information.

    Returns:
        list[dict]: List of assets with only essential fields.
    """
    client = CogniteClient()
    assets = client.data_modeling.instances.list(
        sources=[CogniteAsset.get_source()],
        limit=10
    )

    # Extract only relevant fields
    return [
        {
            "external_id": asset.external_id,
            "name": asset.name,
            "description": asset.description
        }
        for asset in assets
    ]
```

As a last resort, you can use the `.dump()` method to serialize all information from Cognite SDK objects, but this may include unnecessary data that increases token usage.

```
    client = CogniteClient()
    assets = client.data_modeling.instances.list(sources=
[CogniteAsset.get_source()])
    return assets[0].dump()  # ✅ Correct - JSON serializable

def handle() -> list[dict]:
    """Get all assets using dump()."""
    from cognite.client import CogniteClient
    from cognite.client.data_classes.data_modeling.cdm.v1 import CogniteAsset

    client = CogniteClient()
    assets = client.data_modeling.instances.list(sources=
[CogniteAsset.get_source()])
    return assets.dump()  # ✅ Correct - JSON serializable
```

Best practices

We recommend the following best practices when writing Python code for the **Run Python code** tool.

Error handling

Use descriptive error messages:

```
    Args:
        value (float): Input value to process.

    Returns:
        float: Processed value.
    """
    if value < 0:
        raise ValueError("The value must be non-negative.")  # Clear,
actionable message

    if value > 1000:
        raise ValueError("The value exceeds the maximum allowed limit of
1000.")

    return value * 2
```

Documentation

- Always include docstrings following the **Google Python Style Guide**.
- **Parameter descriptions are critical**: Only the `Args:` section descriptions are passed to the AI agent as part of the JSON schema.
- Write clear, specific parameter descriptions that help the agent understand what values to provide.
- Include examples in the parameter descriptions when helpful.

Example of parameter documentation for the agent:

```
    Args:
        asset_id (NodeId): The specific asset to analyze. Provide both space
and externalId.
        threshold (float): The performance threshold value in percentage (0-
100).

    Returns:
        dict: The analysis results with performance metrics.
    """
    # Implementation here
    pass
```

Testing

Write comprehensive test functions. You can use both synchronous and asynchronous test functions:

```
    assert result1 == expected_value, "Normal case failed"

    # Test edge cases
    result2 = handle("", 0)
    assert result2 is not None, "Edge case failed"

    # Test error conditions
    try:
        handle("invalid", -1)
        assert False, "Should have raised ValueError"
    except ValueError:
        pass  # Expected

    # Return a sample result for display
    return handle("sample", 100)
```

Troubleshooting

Below are some common errors you might encounter when using the **Run Python code** tool, and how to fix them.

"Function 'handle' not found"

- Ensure your function is named `handle`.
- Check for indentation errors.

"Missing type annotation for parameter(s)"

- Make sure all parameters have type annotations.
- Use supported types only.

"Unsupported type for parameter"

- Use only **supported types**.
- Check for typos in type annotations.

Serialization errors

- Use the `.dump()` method on Cognite SDK objects.
- Make sure the return values are serializable to JSON.

Code execution environment capabilities and limitations

The **Run Python code** tool runs in a controlled WebAssembly environment with specific capabilities and limitations:

- **Runtime**: Pyodide (Python 3.11 in WebAssembly).
- **Execution context**: Runs in a web worker for isolation.
- **Performance**: WebAssembly execution may be slower than native Python.
- **Memory**: Limited by browser memory constraints.

Pre-installed packages

The following packages are automatically available:

- **Cognite Python SDK** (`cognite-sdk`): Pre-configured with authentication.
- **Standard library**: Full Python 3.11 standard library.
- **Core scientific packages** (via Pyodide):

- `datetime` : Date and time handling
- `uuid` : UUID generation

> ⚠ We don't support installing custom packages. This includes:
>
> - `pip install` commands
> - Dynamic package installation
> - Custom or third-party packages not included in Pyodide

If you need custom packages or more advanced functionality, consider using **Cognite Functions** instead. Functions run on Cognite's cloud infrastructure and support:

- Custom package installation via `requirements.txt`
- Longer execution times
- More memory and compute resources
- Full Python ecosystem access

System limitations

The execution environment has certain constraints that you should be aware of when writing your scripts.

- **File system**: No access to local file system or file I/O operations. The scripts have access to a virtual memory-based file system.
- **Network access**: Only API calls to Cognite Data Fusion APIs are supported. Network calls to other servers might get blocked.
- **Process control**: No subprocess or system command execution.
- **Threading**: Limited threading capabilities in WebAssembly environment.
- **Graphics**: Matplotlib backend set to 'AGG' (non-interactive).

1. **JSON schema as tool descriptor**: The generated JSON schema is passed to the AI agent's language model as a tool descriptor. This tells the agent exactly what parameters are available, their types, and what they're for.
2. **Parameter descriptions guide the agent**: The parameter descriptions from your docstring's `Args:` section are included in the schema and used by the agent to understand what values to provide.
3. **Type validation**: The schema ensures the language model provides correctly typed arguments that match your function signature.
4. **Runtime execution**: When the agent calls your tool, the arguments are validated against the schema and then transformed (e.g., NodeId objects are created from dictionaries) before being passed to your `handle()` function.

> Your parameter docstrings are critical. They directly influence how the agent understands and uses your tool.

Last modified on January 30, 2026

Was this page helpful? Yes No

Agent tools library
Previous

Call Function
Next

Cognite Docs

Resources

Academy

Community

Support

Company

About us

Careers

Blog

Status

Cognite services